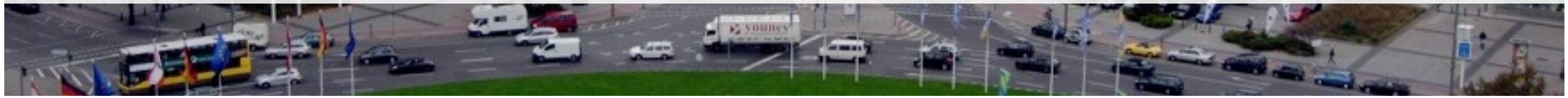


MODUL KÜNSTLICHE INTELLIGENZ: GRUNDLAGEN UND ANWENDUNGEN

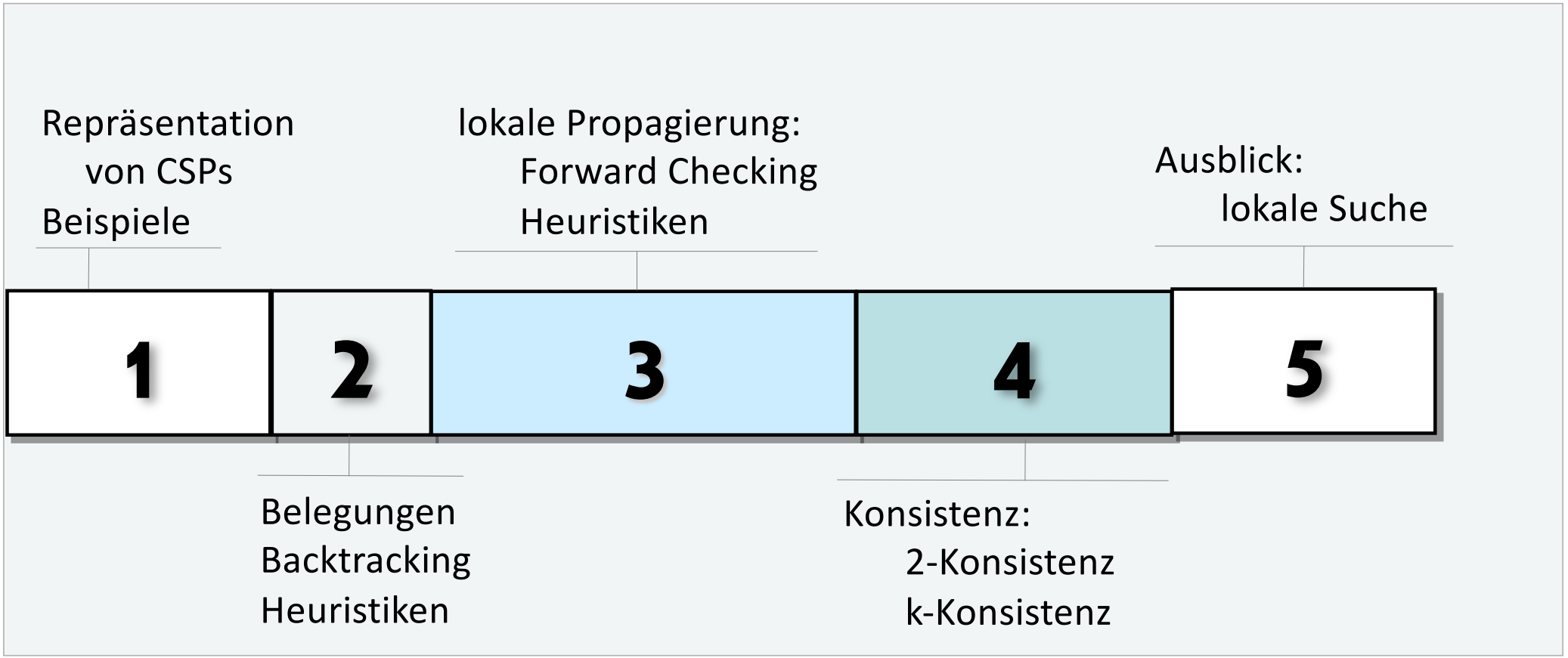
CONSTRAINT SATISFACTION PROBLEMS (CSP)

Einleitung | Backtracking | Forward Checking | Heuristiken | Zweikonsistenz | lokale Suche | Ausblick

Dr.-Ing. Stefan Fricke | Fachgebiet *Service-centric Networking* | TU Berlin



Agenda Constraint Satisfaction Problems (CSPs)



Ein Constraint Satisfaction Problem (CSP) besteht aus Domainvariablen und Constraints

Variablen

x_i mit nichtleerem Wertebereich D_{x_i}

notiert als: $Typ \in \{ gerade,\ ungerade,\ prim \}$,

oder: $D_{Tage} = \{ 1, \dots, 31 \}$

Finite Domain CSPs : endlich abzählbare Wertebereiche

Constraints

C_j zwischen Variablen $x_k, \dots, x_m$ ist Relation $R \subseteq D_k \times \dots \times D_m$

Beispiele:

$x \leq 4$ (einstellig)

$x + y = z$ (dreistellig)

$C(Zahl, Typ)$: $\{ (3, ungerade), (3, prim), (4, gerade) \}$ (zweistellig)

Sudoku als Constraint Satisfaction Problem

9 x 9 Variablen:

$v(a,1), \ldots, v(i,9) \in \{1,2,\ldots,9\}$

Constraints:

$all - different(\{ v(a,7), v(a,8), v(a,9), v(b,7), \ldots, v(c,9) \}),$

…

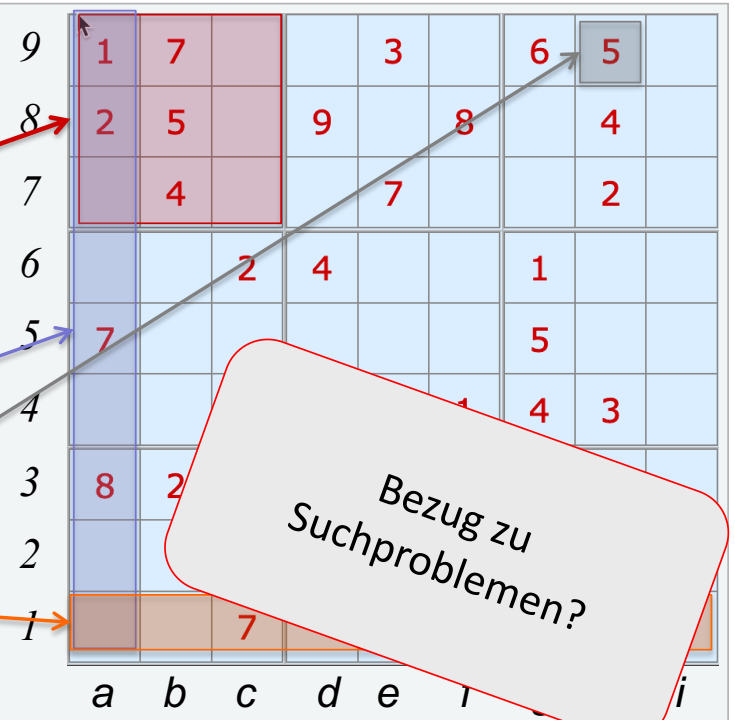
$all - different(\{v(a,1), v(a,2), \ldots, v(a,9)\}),$

…

$all - different(\{v(a,1), v(b,1), \ldots, v(i,1)\}),$

…

$v(a,3) = 8, v(a,5) = 7, \ldots, v(h,9) = 5.$



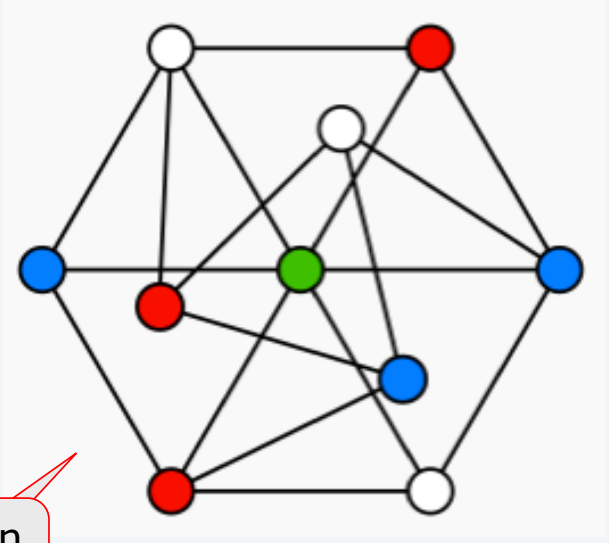
$all - different(Vars) = \forall x, y \in Vars : x \neq y$, wenn x und y verschiedene Variablen sind

Graphenfärbung ist ein Standardproblem der KI

Sudoku, Klausurterminplanung, ...

Färbe die Knoten eines Graphen mit maximal x Farben, sodass benachbarte Knoten unterschiedlich gefärbt sind.

Variablen:	Knoten
Domains:	{rot, blau, ...}
Constraints:	für alle mit einer Kante verbundenen Knoten i, j: $i \neq j$

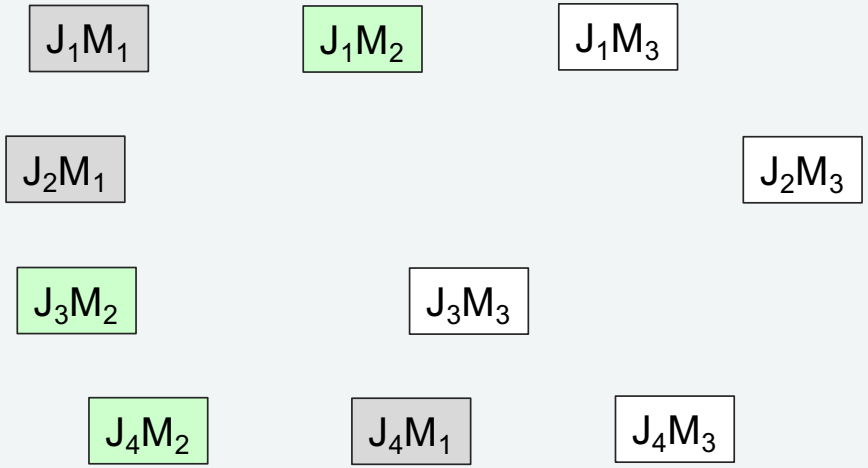


Bestimmung der chromatischen Zahl ist NP-vollständig

Bildquelle: wikipedia.de, CC BY-SA 3.0

Job Shop Scheduling als Constraintproblem

Job	1. Schritt	2. Schritt	3. Schritt
J_1	M_1 (4h)	M_2 (3h)	M_3 (5h)
J_2	M_1 (8h)	M_3 (2h)	-
J_3	M_2 (4h)	M_3 (1h)	-
J_4	M_2 (6h)	M_1 (2h)	M_3 (2h)



Variablen ~ Aktivitäten

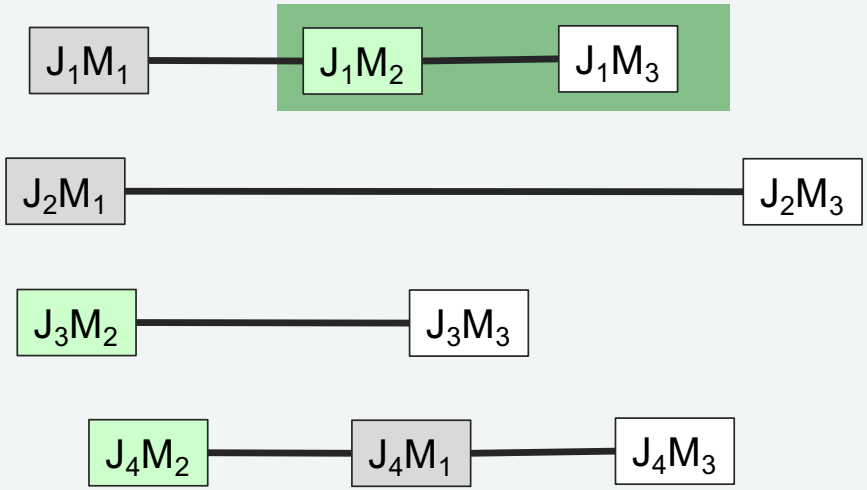
J_1M_1, J_2M_1, ..., J_4M_3

Domains ~ Startzeitpunkte

{ 0, ..., 20 }

Vorher-Nachher-Beziehungen sind binäre Constraints

Job	1. Schritt	2. Schritt	3. Schritt
J_1	M_1 (4h)	M_2 (3h)	M_3 (5h)
J_2	M_1 (8h)	M_3 (2h)	-
J_3	M_2 (4h)	M_3 (1h)	-
J_4	M_2 (6h)	M_1 (2h)	M_3 (2h)



Variablen ~ Aktivitäten

$J_1M_1, J_2M_1, ..., J_4M_3$

Domains ~ Startzeitpunkte

{ 0, ..., 20 }

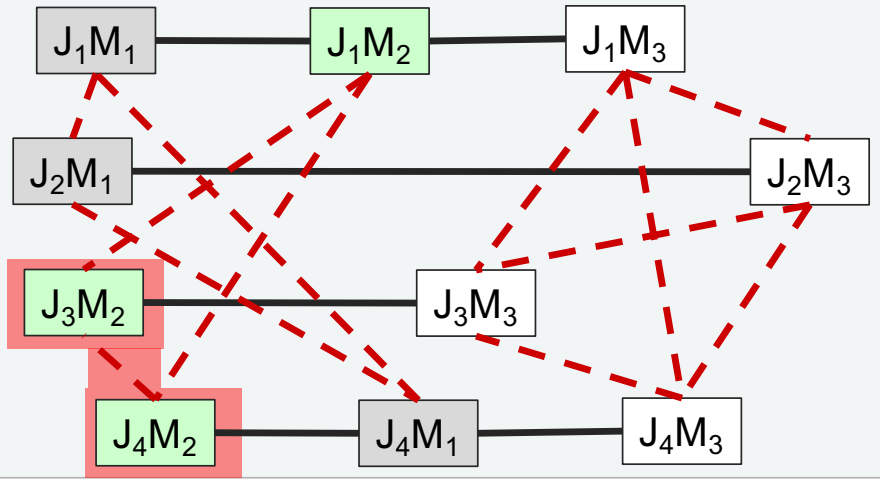
Constraints:

Vorher-Nachher-Beziehungen für Jobs

——— : $J_1M_2 + 3 \leq J_1M_3$

Überlappungsfreiheit ist ein komplexeres binäres Constraint

Job	1. Schritt	2. Schritt	3. Schritt
J_1	M_1 (4h)	M_2 (3h)	M_3 (5h)
J_2	M_1 (8h)	M_3 (2h)	-
J_3	M_2 (4h)	M_3 (1h)	-
J_4	M_2 (6h)	M_1 (2h)	M_3 (2h)



Variablen ~ Aktivitäten

$J_1M_1, J_2M_1, ..., J_4M_3$

Domains ~ Startzeitpunkte

$\{ 0, ..., 20 \}$

Constraints:

Vorher-Nachher-Beziehungen für Jobs

——— : $J_1M_2 + 3 \leq J_1M_3$

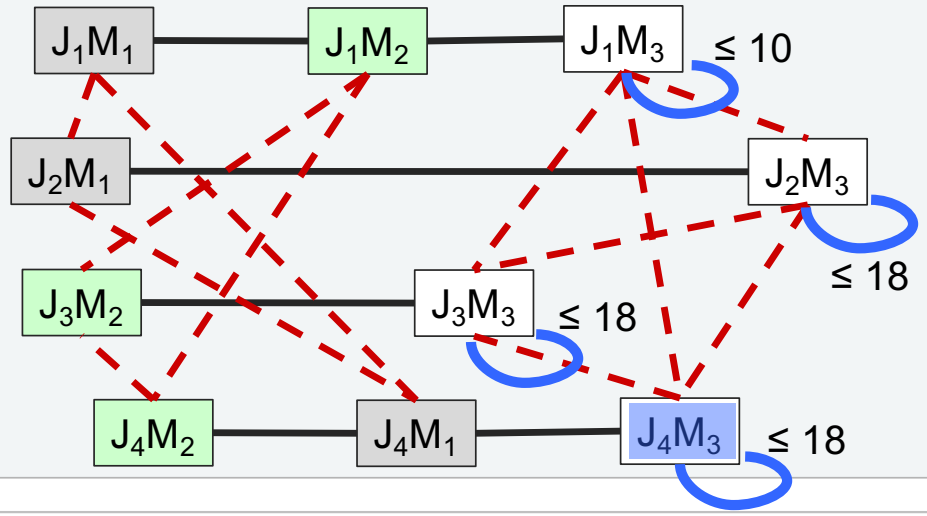
Überlappungsfreiheit für Maschinen

- - - :

$(J_3M_2 + 4 \leq J_4M_2) \vee (J_3M_2 \geq J_4M_2 + 6)$

Termine sind einstellige Constraints

Job	1. Schritt	2. Schritt	3. Schritt
J_1	M_1 (4h)	M_2 (3h)	M_3 (5h)
J_2	M_1 (8h)	M_3 (2h)	-
J_3	M_2 (4h)	M_3 (1h)	-
J_4	M_2 (6h)	M_1 (2h)	M_3 (2h)



Variablen ~ Aktivitäten

$J_1M_1, J_2M_1, ..., J_4M_3$

Domains ~ Startzeitpunkte

$\{ 0, ..., 20 \}$

Constraints:

Vorher-Nachher-Beziehungen für Jobs

——— : $J_1M_2 + 3 \leq J_1M_3$

Überlappungsfreiheit für Maschinen

- - - :

$(J_3M_2 + 4 \leq J_4M_2) \vee (J_3M_2 \geq J_4M_2 + 6)$

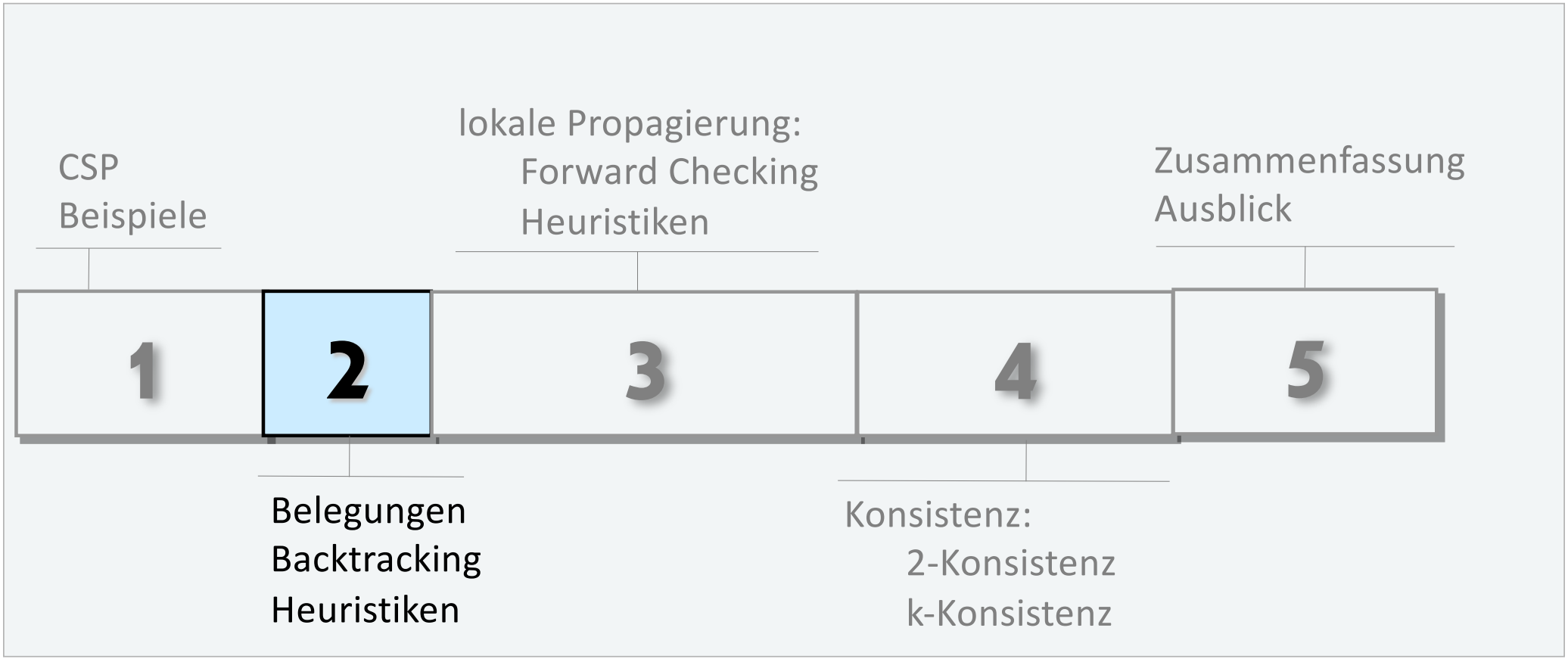
Termine : $J_4M_3 + 2 \leq 20$

Quizfrage

Welches Suchverfahren ist für CSPs am besten geeignet?

a) Breitensuche
b) Uniforme Kostensuche
c) Tiefensuche
d) iterative Tiefensuche
e) A*

Agenda Constraint Satisfaction Problems (CSPs)



Ein CSP wird durch konsistentes Belegen aller Variablen gelöst

a) Belegung (= Zustand)

Zuweisung von Werten
aus D_i an 0,...,i Variablen x_i

b) Konsistente Belegung

Belegung, die keine Constraints verletzt

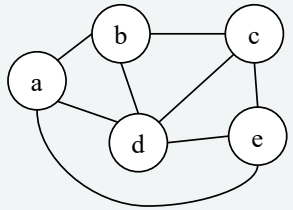
c) Vollständige Belegung

Belegung aller Variablen x_i mit Werten aus D_i

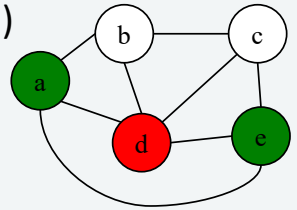
d) Lösung des CSP

Vollständige und konsistente Belegung

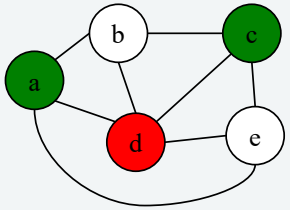
a1)



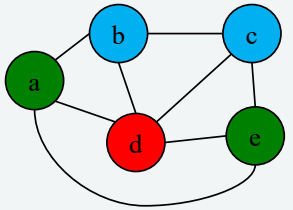
a2)



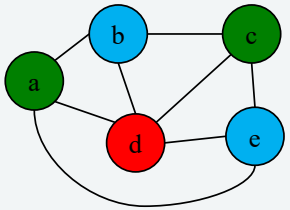
b)



c)



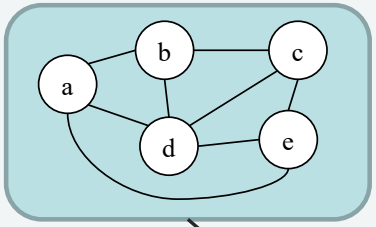
d)



keine Optimierung!

CSPs sind Suchprobleme mit uniformen Aktionskosten

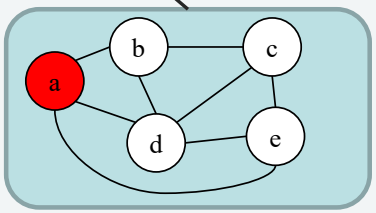
Startzustand: Leere Belegung



$D_a = ... = D_e = \{r,g,b\}$

Aktion: Belege eine freie Variable

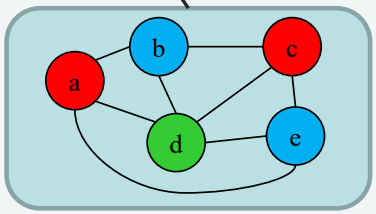
a = r



Jede Variable wird auf dem Lösungspfad exakt einmal belegt

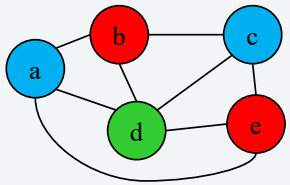
...

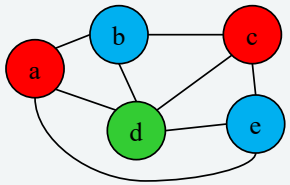
Zieltest: Vollständige konsistente Belegung



Besonderheiten beim Lösen von CSPs

Jede Lösung ist gleich gut



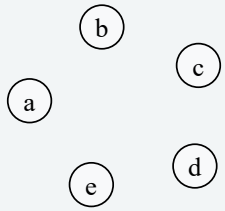


Keine Optimierung z.B. beim Job Shop Scheduling

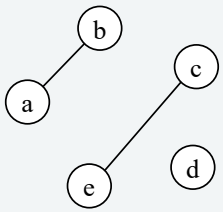
Ausblick:

Stundenplan- Schichtplan- und Fahrplanerstellung mit Softconstraints und Constraintoptimierung

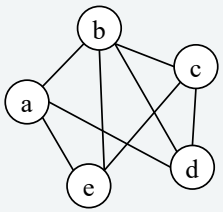
Von einfachen und schwierigen Constraintproblemen



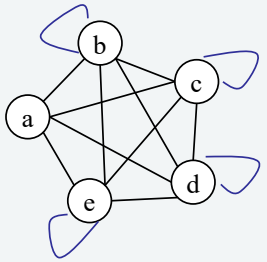
trivial



einfach

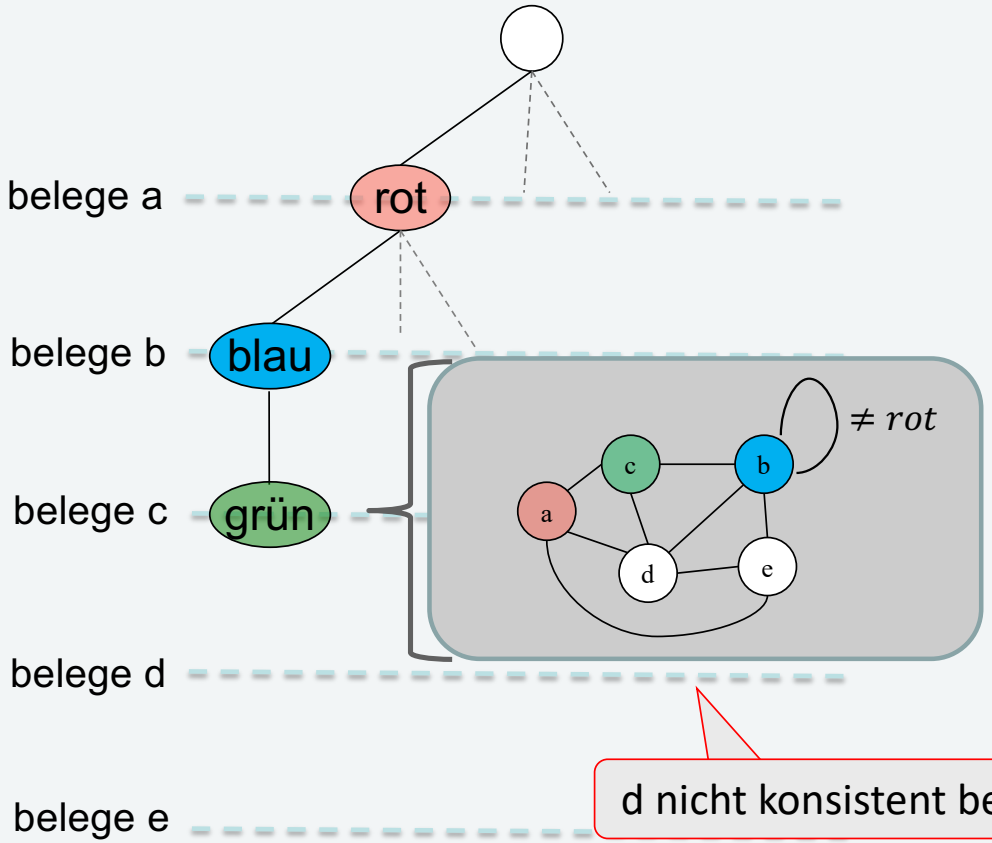


schwierig



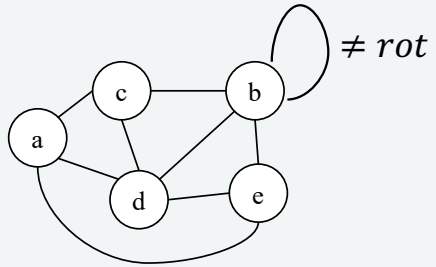
einfach
(overconstrained)

Backtrackingsuche verzahnt das Belegen und Testen



$D_x = \{ rot, blau, grün \} \forall x \in \{a, b, c, d, e\}$

$a \neq c, a \neq d, a \neq e, b \neq rot, b \neq c, b \neq d, b \neq e, d \neq e$



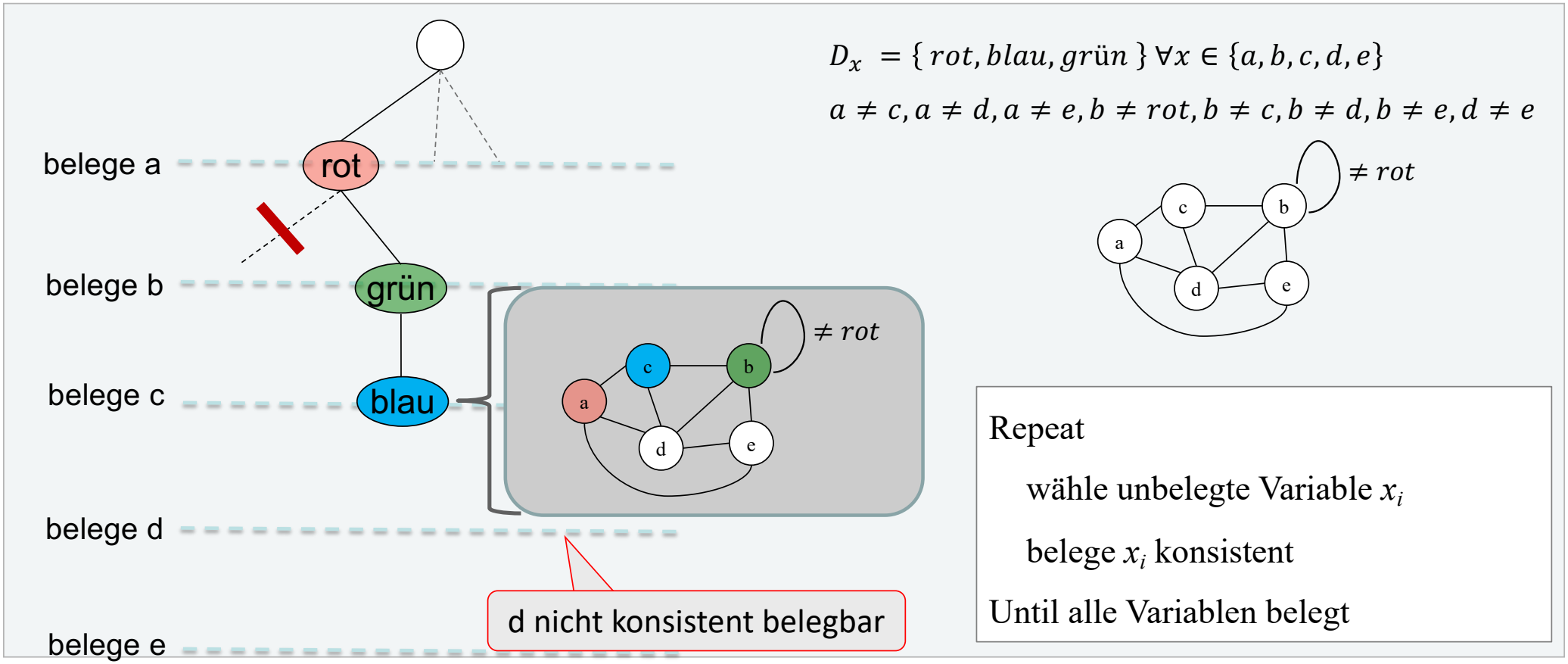
Repeat

wähle unbelegte Variable x_i

belege x_i konsistent

Until alle Variablen belegt

Fortsetzung der Suche nach dem 1. Backtrackingschritt in Baumebene 1



$D_x = \{ rot, blau, grün \} \forall x \in \{a, b, c, d, e\}$

$a \neq c, a \neq d, a \neq e, b \neq rot, b \neq c, b \neq d, b \neq e, d \neq e$

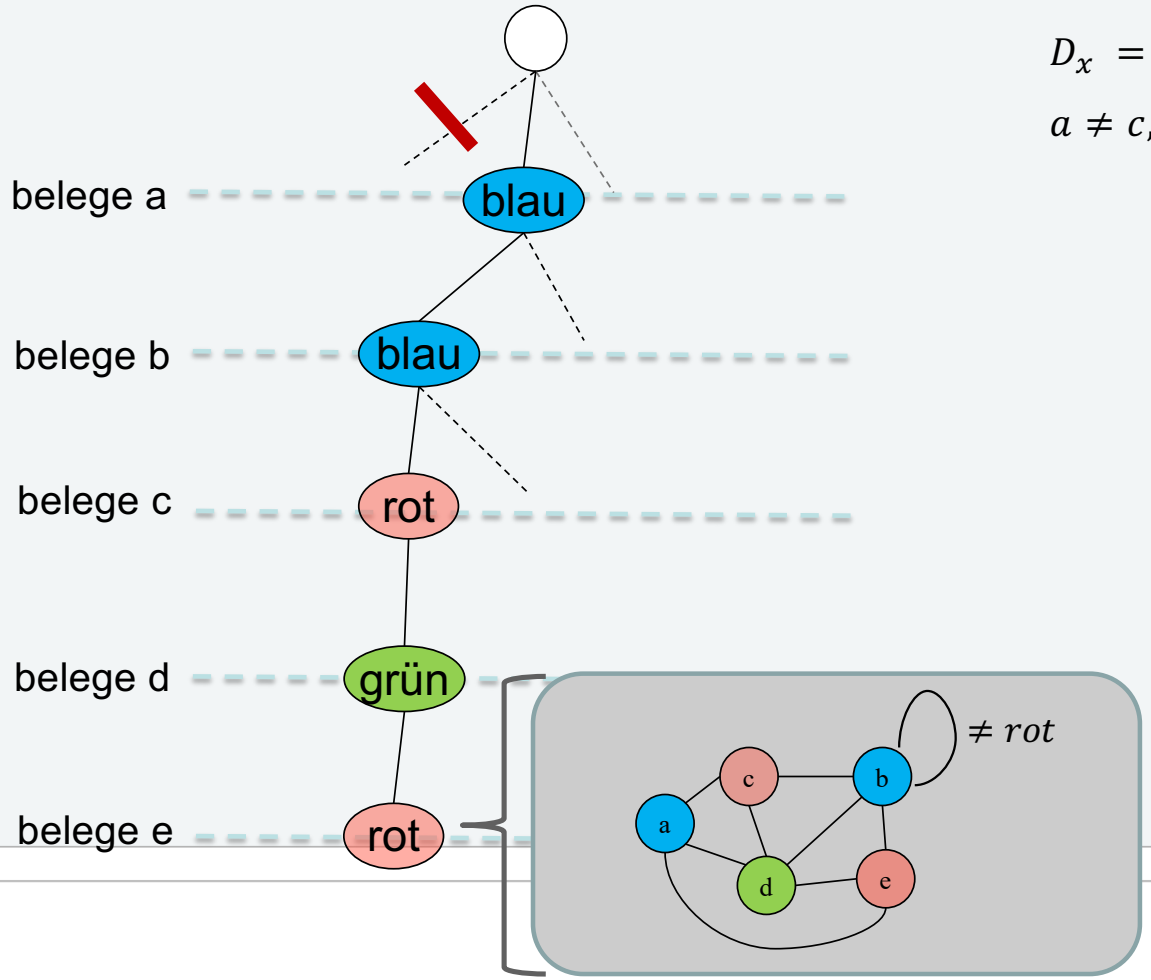
Repeat

wähle unbelegte Variable x_i

belege x_i konsistent

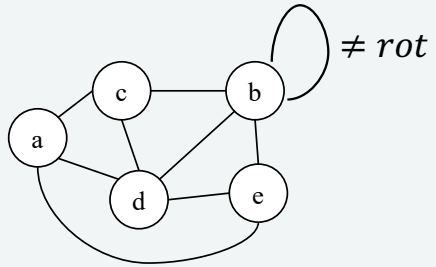
Until alle Variablen belegt

Fortsetzung der Suche nach dem 2. Backtrackingschritt in Baumebene 0



$D_x = \{ rot, blau, grün \} \forall x \in \{a, b, c, d, e\}$

$a \neq c, a \neq d, a \neq e, b \neq rot, b \neq c, b \neq d, b \neq e, d \neq e$



Repeat

wähle unbelegte Variable x_i

belege x_i konsistent

Until alle Variablen belegt

Backtracking-Algorithmus mit Stack

1. Form a 1-element stack consisting of an empty assignment of the variables.
2. Until the first assignment in the stack is complete (and therefore consistent) or the stack is empty
3. Remove the first assignment from the stack.
4. Create new assignments by assigning a value to a single unassigned variable in all possible ways.
5. Reject all inconsistent assignments.
6. Push the new assignments, if any, to the top of the stack.
7. If a complete (and therefore consistent) assignment is found, return assignment;
   otherwise announce failure.

Backtracking hat gute Eigenschaften für CSPs – bis auf die Zeitkomplexität

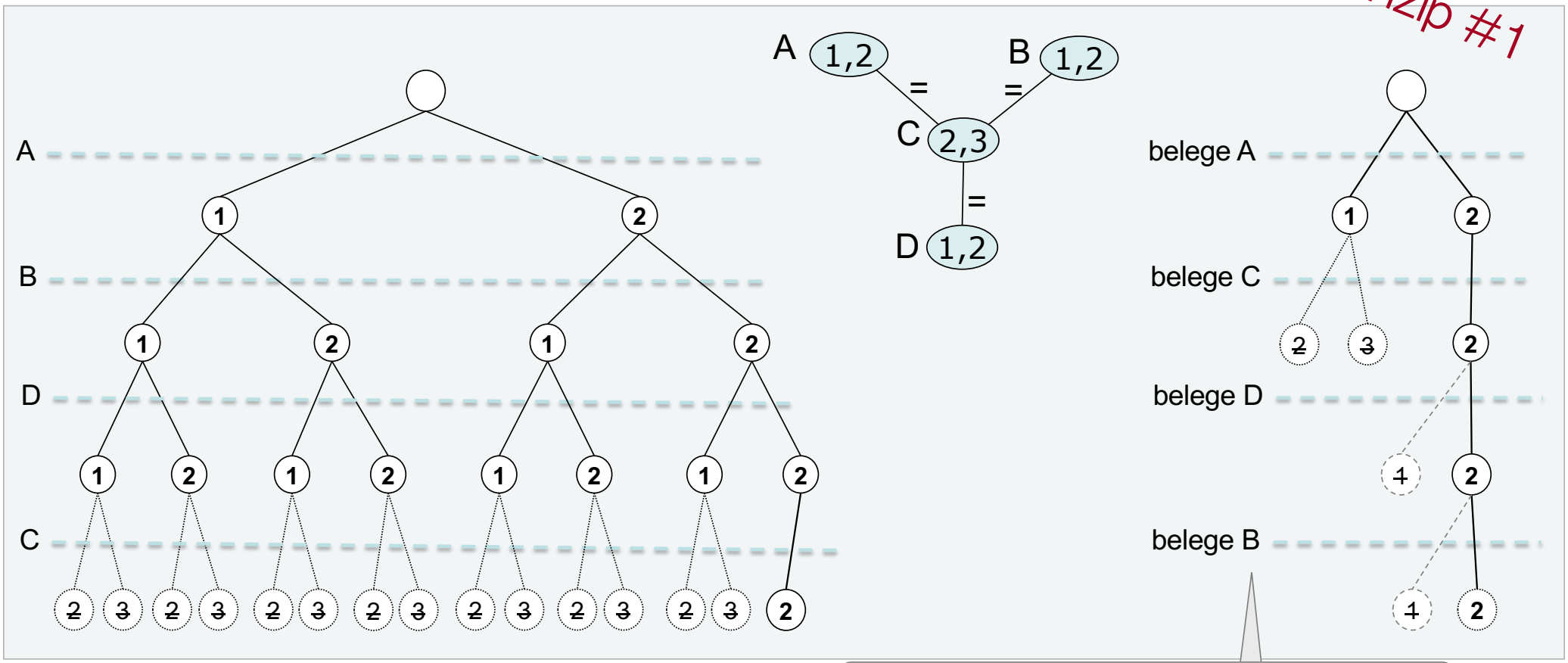
b: maximale Wertebereichsgröße

n: Anzahl der Variablen

Vollständigkeit	Ja	
Optimalität	Ja	(zulässige Lösung genügt)
Zeitkomplexität	$O\,(b^n)$	
Speicherkomplexität	$n^2\,b^2$	(binäre Constraints)

Frühe Konsistenzchecks begrenzen den Suchbaum

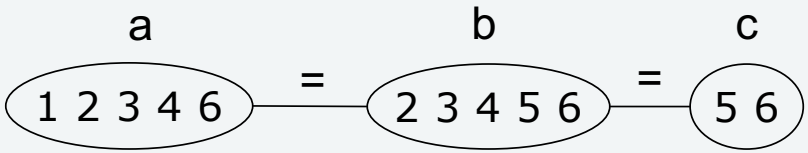
First Fail Prinzip #1



Belege Variable mit den meisten Constraints zu bereits belegten Variablen

Kleine Wertebereiche machen das Scheitern wahrscheinlicher

First Fail Prinzip #2



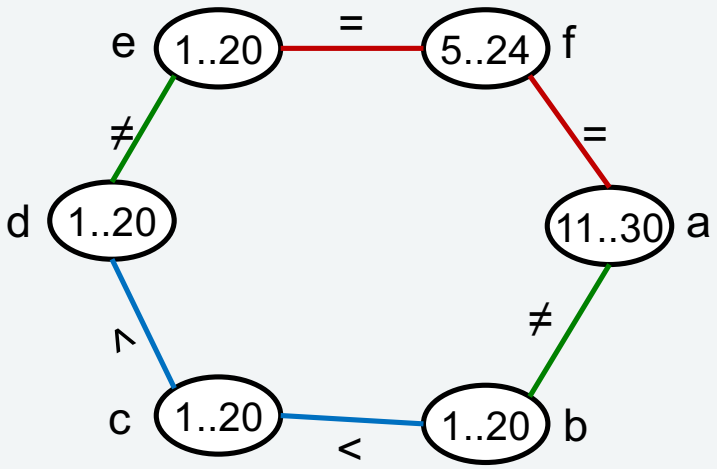
a=1
a=2, b=2
a=3, b=3
a=4, b=4
a=6, b=6, c=6

Belege Variable mit kleinstem Wertebereich zuerst

c=5, b=5
c=6, b=6, a=6

Stark einschränkende Constraints zuerst auswerten

First Fail Prinzip #3

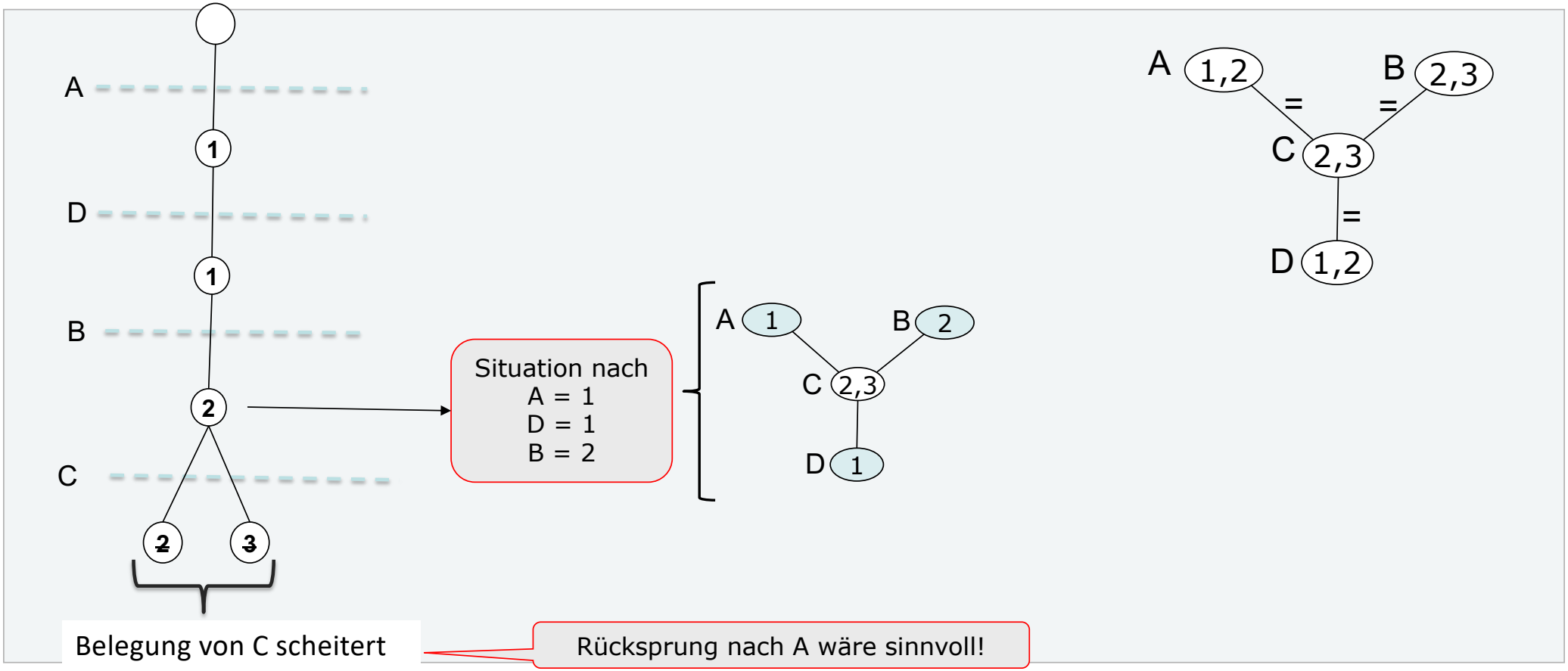


Reihenfolge: „kleine“ Relationen vor großen:

$$= \; vor \; \{<, >\} \; vor \; \{\leq, \geq\} \; vor \; \neq$$

eine gute Belegungsreihenfolge: e, f, a, b, c, d

Chronologisches Backtracking leidet unter „dummen“ Rücksprüngen



Lokale Constraintpropagierung / partial look ahead



Lösen von Sudokus mittels Forward Checking (und Backtracking)

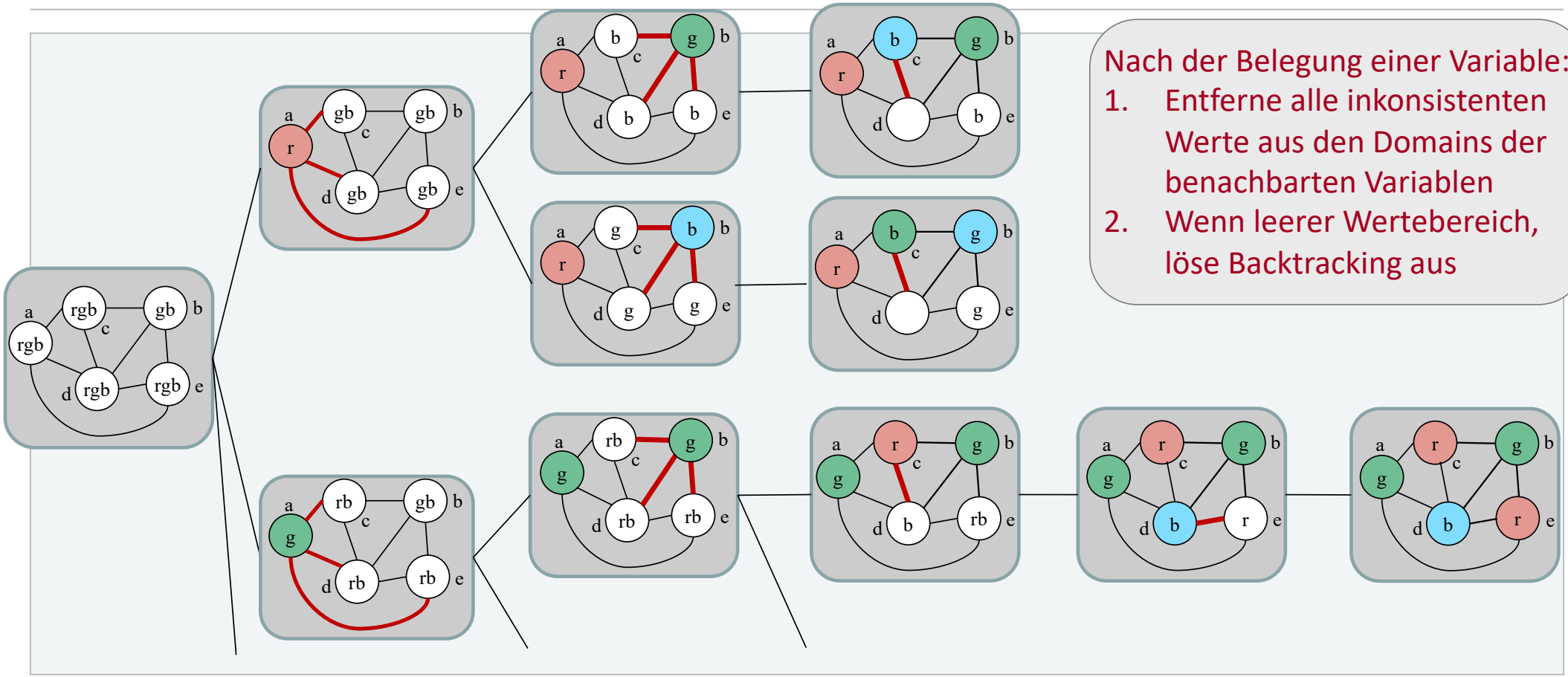
”Intelligentere” Backtrackingsuche

Verwaltung aller Variablen und ihrer Domains im Zustand

1	7	8 9	2	3	2 4	6	5	8 9
2	5	3 6	9	1 6	8	3 7	4	1 7
3 6 9	4	3 6 8 9	1 5 6	7	5	3 8 9	2	1 8 9
3 6 9	3 6 8 9	2	4	5 6 8 9	3 5 7 9	1	6 7 8 9	6 7 8 9
7	1 3 6 8 9	1 3 4 6 8 9	2 3 6	2 6 8 9	2 3 9	5	6 8 9	2 6 8 9
6 9	6 8 9	5	2 6 7	2 6 8 9	1	4	3	2 6 7 8 9
8	2	1 3 4 6 9	1 3 5 7	1 4 5 9	3 4 5 7 9	7 9	1 6 7 9	1 4 5 6 7 9
3 4 5 6 9	1 3 6 9	1 3 4 6 9	8	1 2 4 5 9	2 3 4 5 7 9	2 7 9	1 6 7 9	1 2 4 5 6 7 9
4 5 9	1 9	7	1 2 5	1 2 4 5 9	6	2 8 9	1 8 9	3

Partial-Look-Ahead-Idee:
nach dem Belegen einer Variable inkonsistente Werte aus den benachbarten unbelegten Variablen entfernen

Backtrackingsuche in Verbindung mit Forward Checking begrenzt den Suchraum

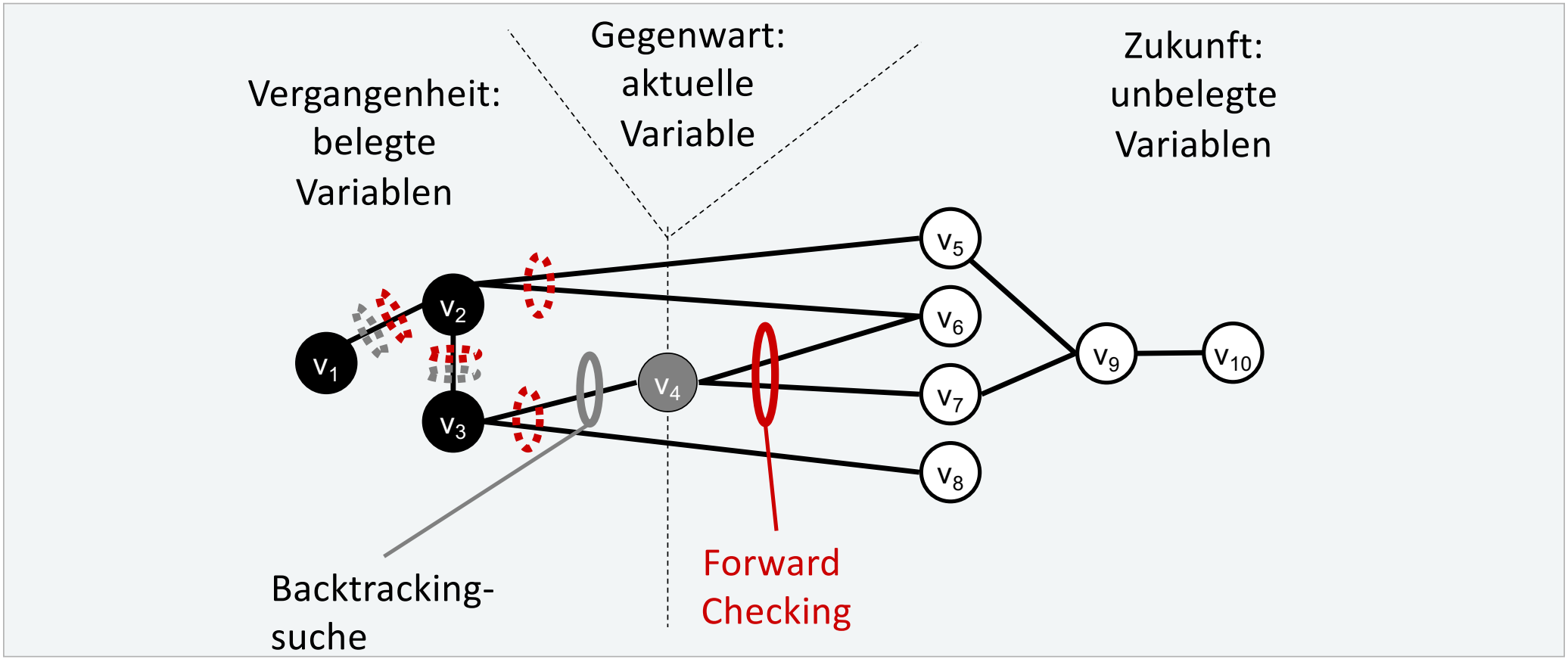


Nach der Belegung einer Variable:

1. Entferne alle inkonsistenten Werte aus den Domains der benachbarten Variablen
2. Wenn leerer Wertebereich, löse Backtracking aus

$$D_x = \{ r, b, g \}\ \forall x \in \{a, c, d, e\}, D_b = \{g, b\}, \qquad a \neq c, a \neq d, a \neq e, b \neq c, b \neq d, b \neq e, d \neq e$$

Forward Checking realisiert ein „partial look ahead“



Integration von Forward Checking in die Backtrackingsuche

1 REPEAT
2 wähle unbelegte Variable x
3 belege x mit Wert $v \in D_x$
4 Entferne für jede unbelegte Variable y, die in einem Constraint $c(x, y)$ vorkommt, alle mit v inkonsistenten Werte
5 Falls $D_y = \{\}$ existiert, löse Backtracking aus
6 UNTIL alle Variablen belegt

Forward Checking hat quadratischen Zeitaufwand

b: größte Wertebereichsgröße, n: Zahl der Variablen

maximal ...

- pro Constraint: b Vergleiche mit belegtem Wert
- n-1 Constraints auswerten

→ Zeitaufwand: $O(n \cdot b)$

Speicheraufwand:
$O(n^2 \cdot b)$

REPEAT

wähle unbelegte Variable x

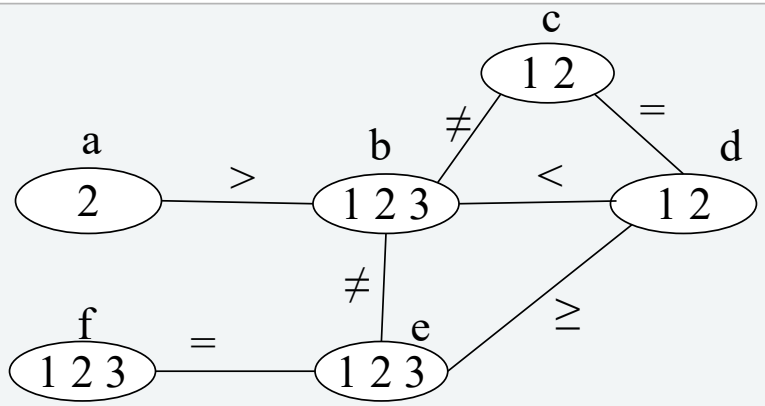
belege x mit Wert $v \in D_x$

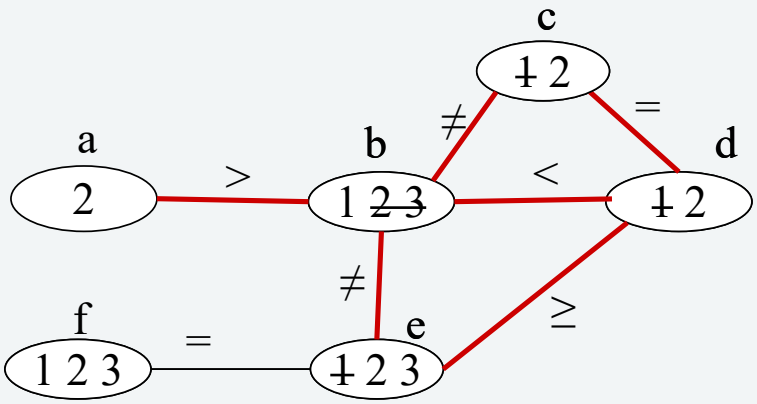
Entferne für jede unbelegte Variable y,
die in einem Constraint $c(x, y)$ vorkommt,
alle mit v inkonsistenten Werte

Falls $D_y = \{\}$ löse Backtracking aus

UNTIL alle Variablen belegt

Aggressives Forward Checking propagiert einstellige Wertebereiche bis zum Stillstand





FC Vars	offene Const.	D_a	D_b	D_c	D_d	D_e	D_f
a	ab, bc, bd, be, cd, de, ef	2	1,2,3	1,2	1,2	1,2,3	1,2,3
b	bc, bd, be, cd, de, ef	2	1	1,2	1,2	1,2,3	1,2,3
c, d	cd, de, ef	2	1	2	2	2,3	1,2,3
d	de, ef	2	1	2	2	2,3	1,2,3
	ef	2	1	2	2	2,3	1,2,3

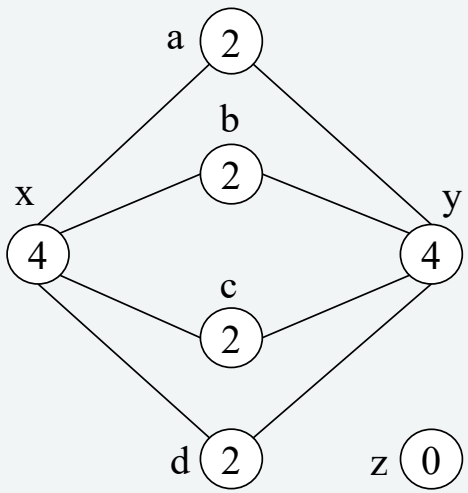
Die Gradheuristik beschleunigt die Backtracking+Forward Checking-Suche

First Fail Prinzip #1

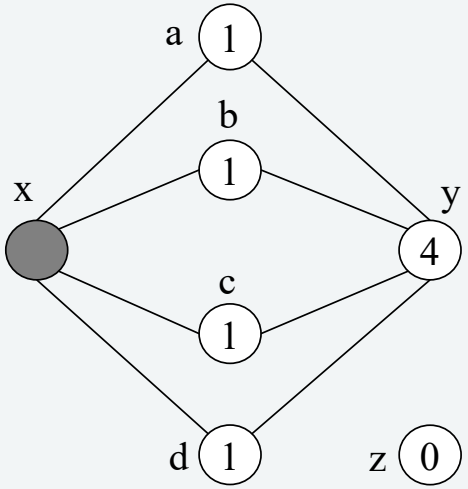
Belege Variable mit den meisten Constraints *zu noch nicht belegten Variablen*.

Grad einer unbelegten Variable = Anzahl offener Constraints

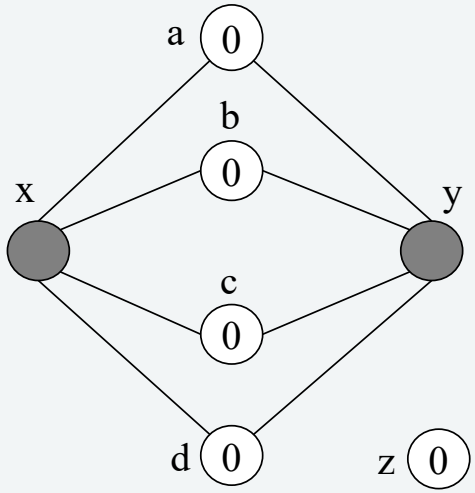
Je höher der Grad, desto weitreichender die Propagierung



Grad der leeren Belegung



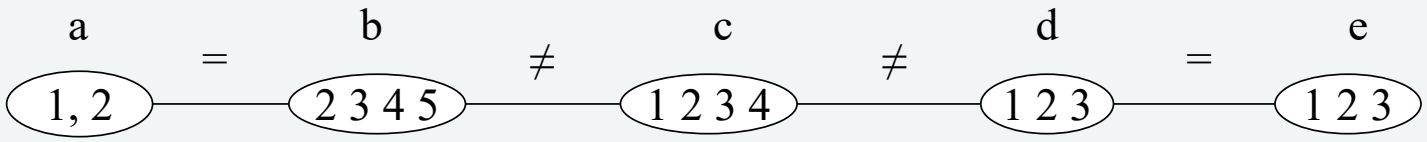
Grad nach Belegung von x



Grad nach Belegung von x und y

Zahl im Knoten = Grad = Anzahl noch ungeprüfter Constraints

Minimum Remaining Values-Heuristik (MRV) minimiert den Verzweigungsgrad des Suchbaums



MRV = Belege Variable mit kleinstem Wertebereich.

Im Initialzustand:

$|D_a| = 2;\ |D_d| = |D_e| = 3;$

$|D_b| = |D_c| = 4$

Tiebreaker: Gradheuristik ⇒

Belegung	MRV	D_a	D_b	D_c	D_d	D_e
	a (2)	{1,2}	{2,3,4,5}	{1,2,3,4}	{1,2,3}	{1,2,3}
a = 1		1		{1,2,3,4}	{1,2,3}	{1,2,3}
a = 2	b (1)	2	{2}	{1,2,3,4}	{1,2,3}	{1,2,3}
b = 2	c, d, e (3)	2	2	{1,3,4}	{1,2,3}	{1,2,3}
d = 1	e (1)	2	2	{3,4}	1	{1}
e = 1	c (2)	2	2	{3,4}	1	1
c = 3		2	2	3	1	1

Heuristiken steigern die Effizienz von Backtracking mit Forward Checking

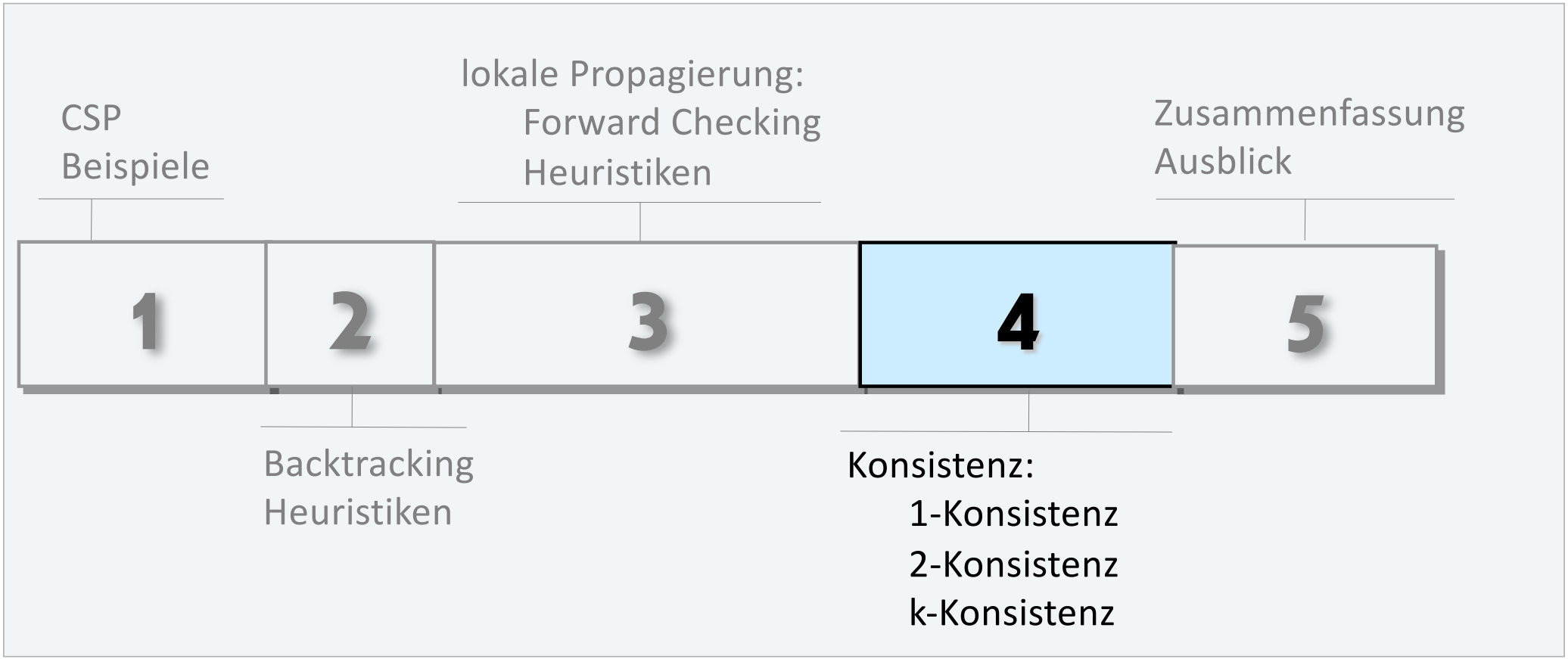
Wahl der nächsten zu belegenden Variable nach ...

- höchstem Grad : Anzahl “offener” Constraints (zu *unbelegten* Variablen)
- Minimum Remaining Values : Variable mit kleinster Domain zuerst
- Stärke der Constraints : stark einschränkende Constraints zuerst (= vor > vor ≥ vor =)

Kombination sinnvoll, z.B.

- Gradheuristik und MRV als Tiebreaker oder
- MRV und Gradheuristik als Tiebreaker

Lokale Konsistenz



Constraintpropagierung *vor* einer Belegung kann die Effizienz der Suche erheblich steigern

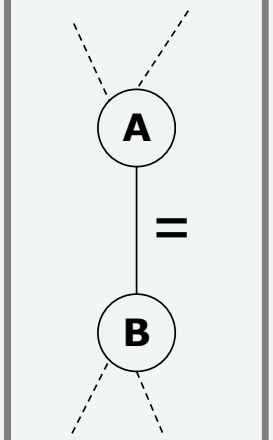
Variablen:

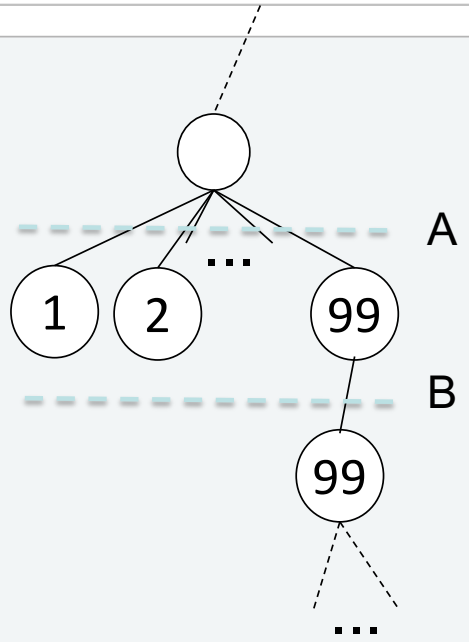
A: {1, ..., 100}

B: {99, ..., 198}

Constraint:

A = B





BT & FC ist ineffektiv! Besser:

zuerst Konsistenz sicher stellen durch Constraintpropagierung,

danach Suche im konsistenten CSP

In einem 1-konsistenten CSP sind alle einstelligen Constraints erfüllt

Sei $C(x)$ das auf x definierte Constraint.

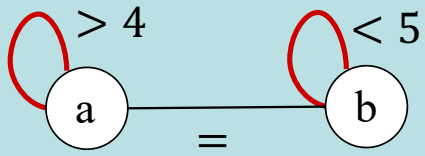
x ist 1-konsistent, wenn $\forall d \in D_x\colon\ d \in C(x)$

CSP ist 1-konsistent, wenn alle Variablen 1-konsistent

Alle 1-stelligen Constraints auswerten: inkonsistente Werte aus Domains entfernen

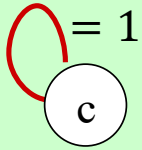
$D_a = \{ 1, 2, 3, 4, 5, 6 \}$

$D_b = \{ 2, 3, 4 \}$



1-konsistent mit: $D_a = \{ 5, 6 \}, D_b = \{ 2, 3, 4 \}$

$D_c = \{ 2, 3, 4 \}$



keine 1-Konsistenz möglich, daher nicht lösbar

Zweikonsistenz betrifft nur binäre Constraints

Sei $C(x_i, x_j)$ ein binäres Constraint.

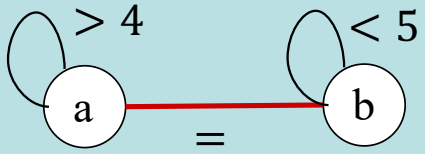
Die Kante (x_i, x_j) ist 2-konsistent, wenn $\forall v \in D_{x_i}\ \exists w \in D_{x_j}: (v, w) \in C(x)$

CSP ist 2-konsistent, wenn alle Kanten 2-konsistent

$C(x_i, x_j)$ enthält 2 Kanten (x_i, x_j) und (x_j, x_i)

$D_a = \{ 1, 2, 3, 4, 5, 6 \}$

$D_b = \{ 2, 3, 4 \}$



2-konsistent mit: $D_a = \{ 2,3,4 \}, D_b = \{ 2,3,4 \}$

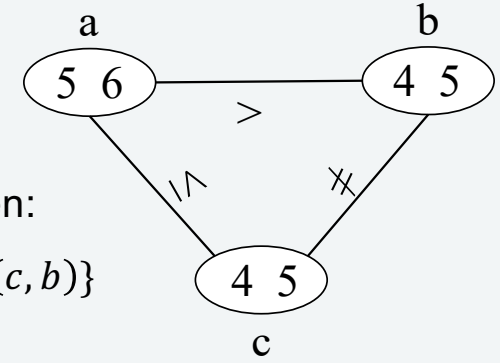
1- und 2-Konsistenz hier nicht herstellbar => unlösbar

starke (strenge) 2-Konsistenz schließt 1-Konsistenz mit ein

Zweikonsistenz betrifft alle Variablenpaare

Jede beliebige Variable x kann mit einem beliebigen Wert $v_x \in D_x$ belegt werden, sodass jede andere Variable y immer noch konsistent belegbar ist.

Einmalige Überprüfung aller Kanten zur Herstellung von 2-Konsistenz reicht nicht aus:

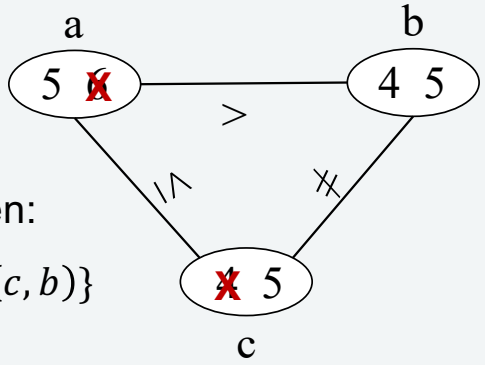


2-konsistente Kanten:

$\{(a,b),(b,a),(b,c),(c,b)\}$

nicht 2-konsistente Kanten:

$\{(a,c),(c,a)\}$



2-konsistente Kanten:

$\{(a,b),(a,c),(c,a),(c,b)\}$

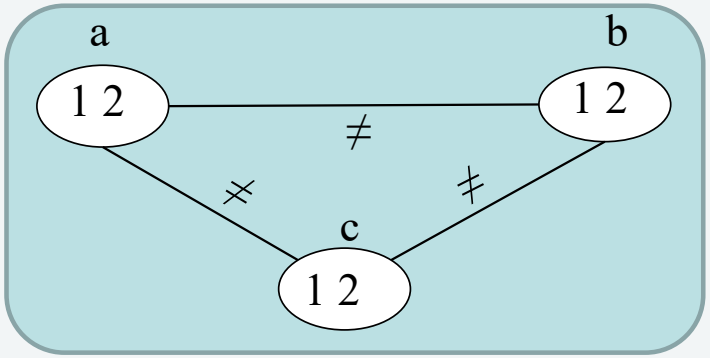
nicht 2-konsistente Kanten:

$\{(b,a),(b,c)\}$

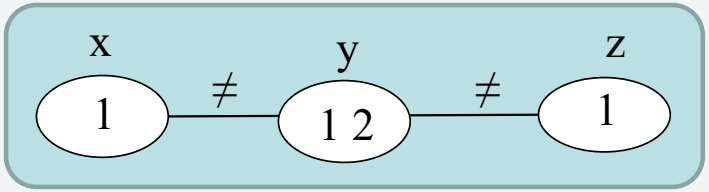
Höhere Grade der Konsistenz

k-Konsistenz:
Für jede konsistente Belegung verschiedener $k-1$ Variablen ist jede k. Variable konsistent belegbar.

Hinreichende Bedingung für Lösbarkeit eines CSPs: starke/strenge **n-Konsistenz**
Problem: Herstellen von k-Konsistenz liegt in $O(b^k)$



2-konsistentes CSP, das nicht in einen
3-konsistenten Zustand überführt werden kann



3-konsistent, aber nicht 2-konsistent

AC-3-Algorithmus zur 2-Konsistenz arbeitet auf gerichteten Zweigen

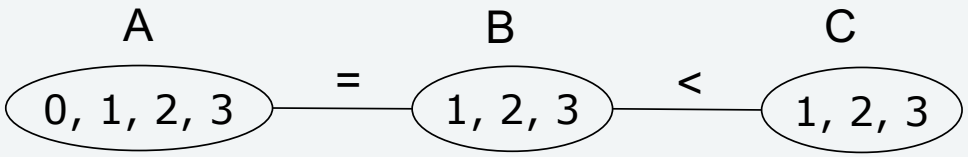
1. procedure AC-3(V, D, C)
2. Q ← create_arcs(C)
3. while not empty Q do
4. remove X→Y from Q
5. D_X' ← arc-reduce(X, Y)
6. if D_X' = {} then return (fail, D_X')
7. if D_X' ≠ D_X then
   Q ← Q ∪ { Z→X | Z≠Y and (c(Z,X)∈C or c(X,Z)∈C) }
8. D_X ← D_X'
9. end while
10. return (true, D)
11. end AC-3

function arc-reduce(X, Y)
  for each x_i in D_X
    if $\neg\exists y_i \in D_y$ such that
      { $X=x_i$, $Y=y_i$ } fulfills c(X,Y)
    then remove x_i from X
  return(D_X)

Queue ergänzen um alle Kanten nach X (außer Y→X)

nach A.K. Mackworth: Consistency in networks of relations. *Artificial Intelligence*, 8, S. 99-118, 1977.

2-Konsistenz mit dem AC-3-Algorithmus berechnen



#	→	D_A	D_B	D_C	enqueue
0		0, 1, 2, 3	1, 2, 3	1, 2 ,3	
1	A → B	1, 2, 3			(%)
2	B → A		k.V.		
3	B → C		1, 2		A → B
4	C → B			2, 3	
5	A → B	1, 2			

2-konsistent mit $D_A = D_B = \{1,2\}; D_C = \{2,3\}$

AC-3 hat polynomiellen Zeitaufwand

Sei

n = die Anzahl der Variablen,

b = die Größe des größten Wertebereichs.

Forward Checking: O(n·b)

maximal n(n-1)/2 Constraints.

Arc-Reduce: maximal b^2 Vergleiche für jede Kante

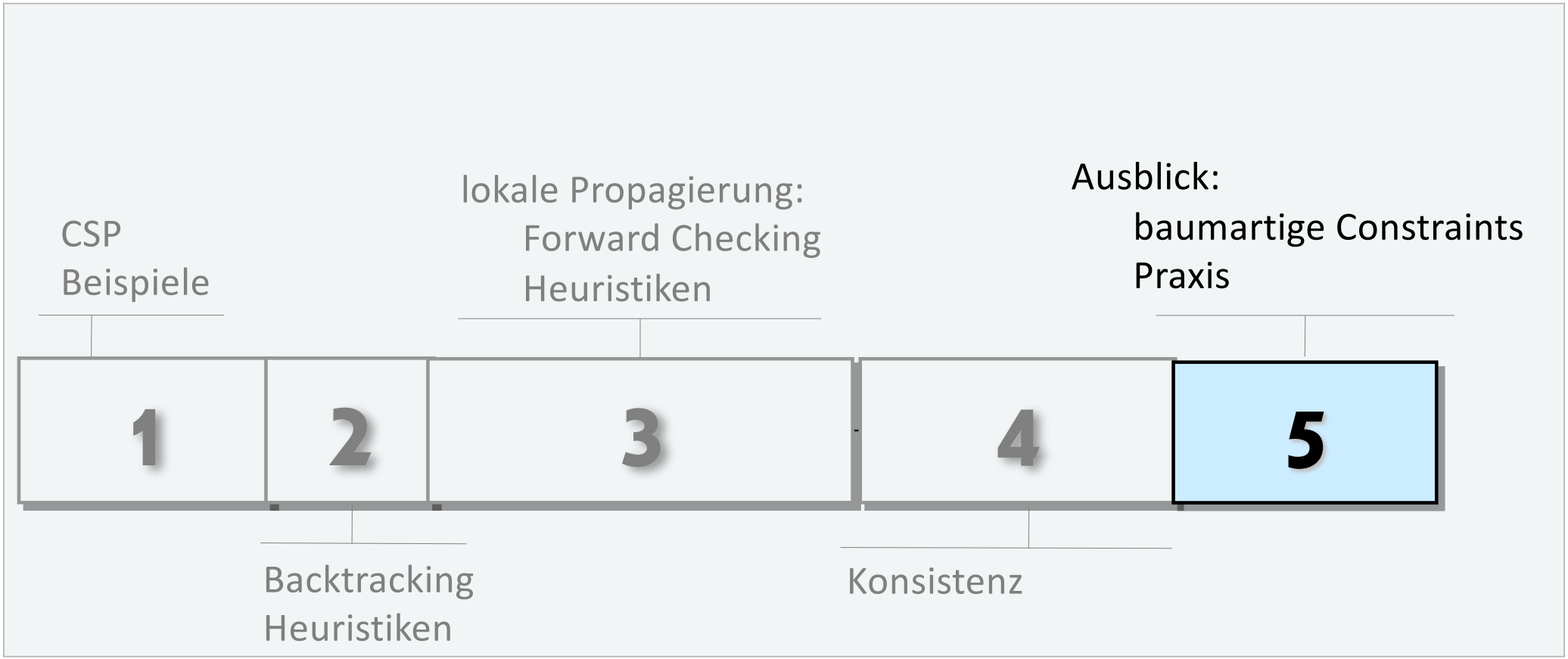
Jede Kante: maximal b mal in Queue.

$O(n^2 \cdot b^3)$

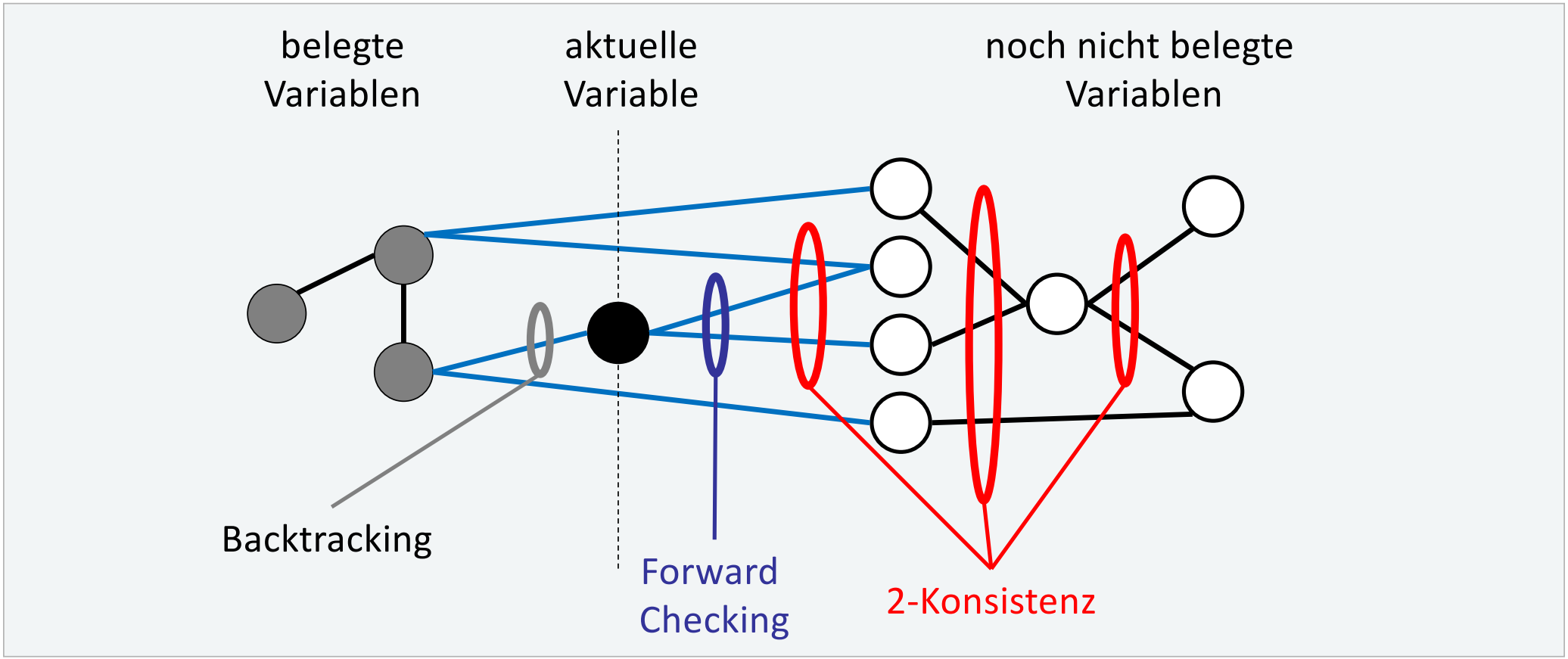
AC-2001: $O(n \cdot b^2)$ (Zeit)
$O(n \cdot b)$ (Speicher)

AC-2001: [Bessiere, et al. 2005]

Zusammenfassung und Ausblick



Die Verfahren im Vergleich



Quelle: http://ktiml.mff.cuni.cz/~bartak/downloads/CPschool05notes.pdf

Fazit

Chronologisches Backtracking: unpraktikabel

Forward Checking = partial look-ahead zumeist effektiver als 2-Konsistenz = full look ahead

2-Konsistenz: Geringerer Suchaufwand versus höherer Rechenaufwand

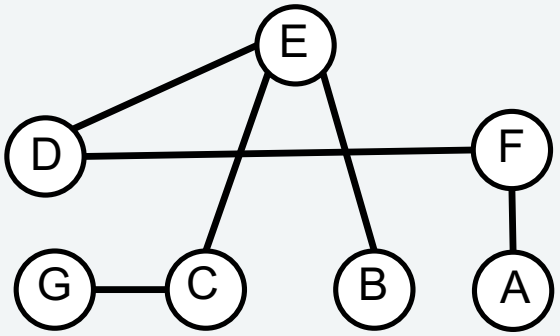
sinnvoll zumeist nur bei stark einschränkenden Constraints

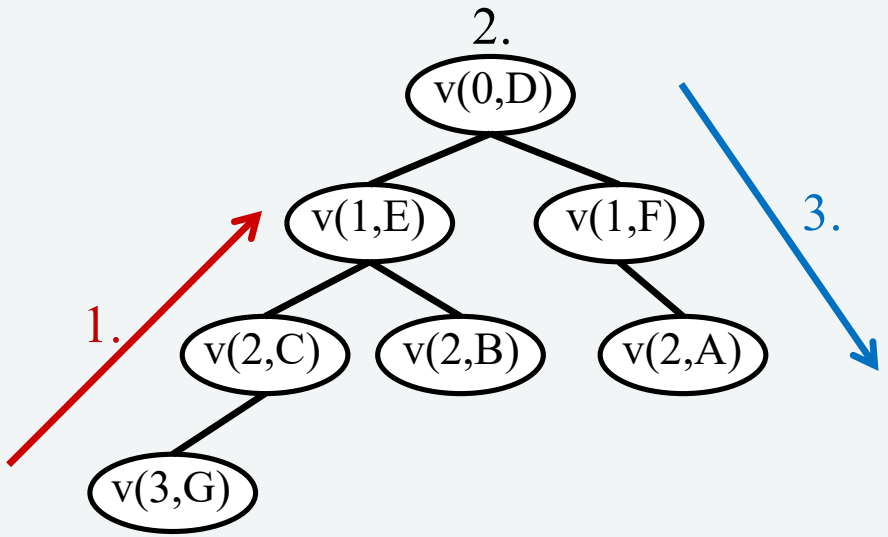
Praxistipp:

1. 1-Konsistenz herstellen
2. gegebenenfalls 2-Konsistenz herstellen
3. Suche mittels Backtracking + Forward Checking unter Einsatz von Heuristiken

Alternative: Lokale Suche … (nächste Woche)

Baumartige CSPs sind in polynomieller Zeit ohne Backtracking lösbar



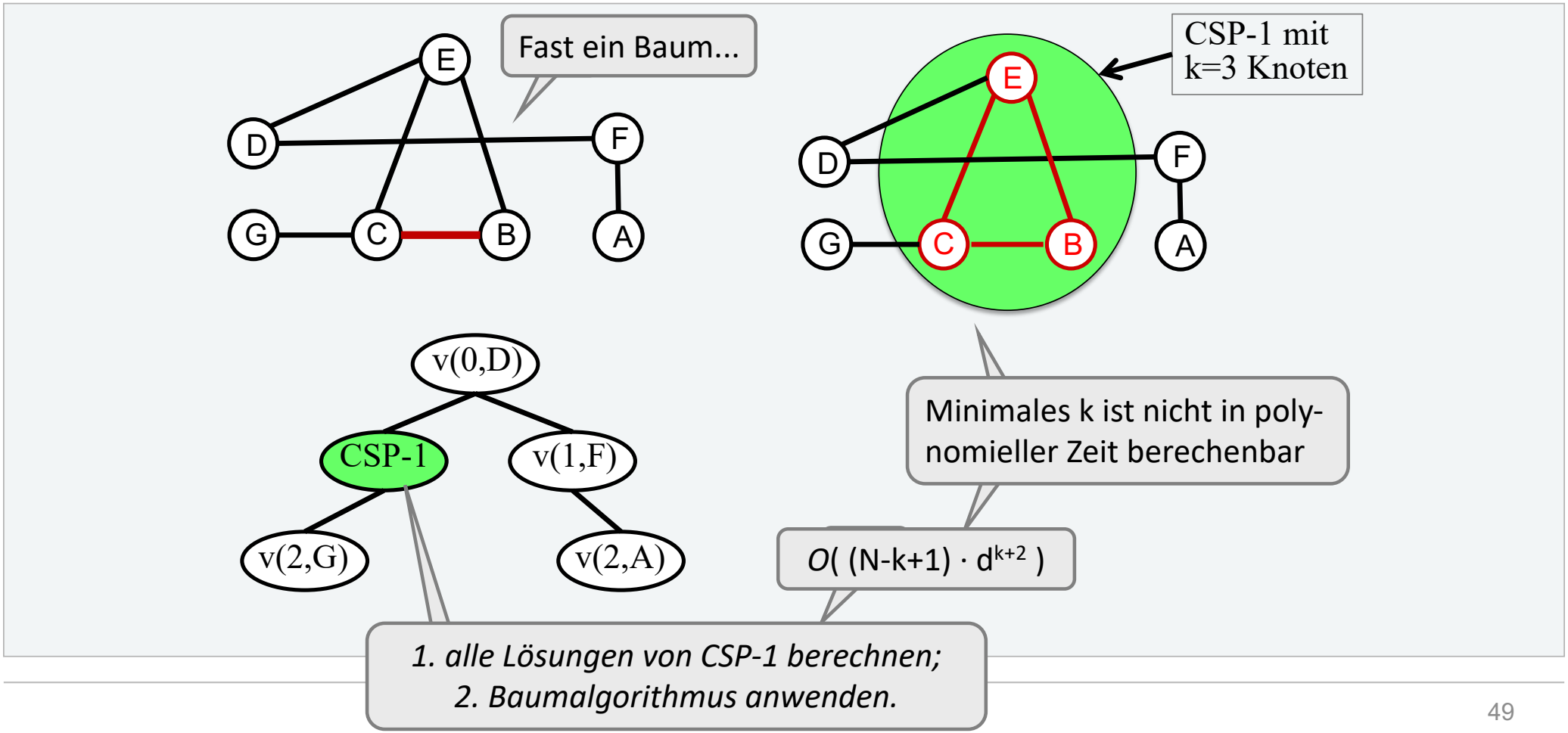


$O(n \cdot d^2)$

1. For i=maxdepth down to 1
   a. entferne aus $D_{V(i-1,k)}$ alle inkonsistenten Werte zu V(i,j)
   b. if $D_{V(i-1,k)} = \{\}$ return failure

2. Belege Wurzelknoten V(0,x)
3. For i=1 to maxdepth
   a. belege V(i,j) konsistent mit Elternknoten

Baumtransformation durch Entfernen echter Graphen reduziert die Komplexität



Ausblick

Constraintbibliotheken und Constraintsolver: auch für Constraintoptimierung

Constraint Logic Programming: deklarative Programmierung

CLP ($\mathbb{R}$) integriert Lineare Programmierung

Constraint Handling Rules: Regelbasierte Auswertung und Vereinfachung



Dr.-Ing. Stefan Fricke

TU Berlin | Fachgebiet *Service-centric Networking*
Ernst-Reuter-Platz 1 | 10587 Berlin | Germany

stefan.fricke@tu-berlin.de

https://twitter.com/kuepp

https://www.xing.com/in/axelkuepper/

http://www.snet.tu-berlin.de/kuepper

